UNIVERSIDAD AUTÓNOMA GABRIEL RENÉ MORENO
FACULTAD DE INGENIERÍA Y CIENCIAS DE LA COMPUTACIÓN Y TELECOMUNICACIONES



LUCES DE NEON

Ciudad interactiva con OpenGL

Estudiante: Melissa Sanchez Cuellar

Materia: Programación Grafica

Docente: Ing. Kenji Kawaida

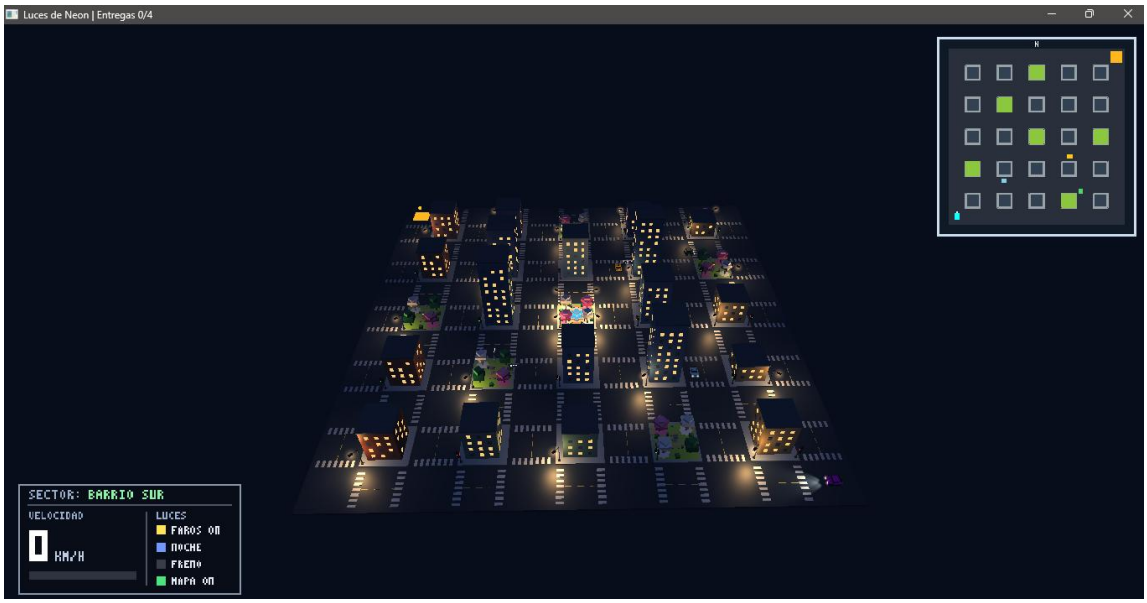
Fecha: 17/10/26

Link: https://github.com/MelissaSanchez20/cuidad-conduccion.git

1. Diseño de la ciudad

Luces de Neón es un juego de conducción en una ciudad 3D construida por código con Java, LWJGL y OpenGL. El jugador maneja un auto por la ciudad y debe completar cuatro entregas en distintos sectores, de día o de noche.

- **Mapa:** 11 × 11 celdas de 10 unidades, con avenidas en ambos sentidos conectadas entre sí.
- **Manzanas:** 25 manzanas: 19 con edificios y 6 parques.
- **Cinco sectores:** Centro, Barrio Norte, Barrio Sur, Distrito Oeste y Distrito Este.
- **Plaza central:** en el centro del mapa, una fuente de agua para que resalte de los demás parques, bancos e iluminarias.
- **Entregas:** el auto parte de la esquina y hay cuatro destinos sobre calles de sectores distintos. Para entregar hay que frenar sobre la marca dorada.



2. Controles

Tecla	Acción
flechas arriba / abajo	Acelerar / frenar y retroceder
flechas izquierdas / derecha	Girar mientras el auto se mueve
Espacio	Freno (enciende las luces de freno)
C	Cambiar entre cámara de seguimiento y cámara aérea
N	Cambiar entre día y noche
F	Encender / apagar los faros
M	Mostrar / ocultar el minimapa
P	Pausa (continuar, reiniciar o salir)
R	Reiniciar el auto, el tráfico y las entregas
Flechas + ENTER	Elegir una opción en los menús de inicio y pausa
ESC	Salir

3. Funciones implementadas

Iluminación

- **Día y noche** con la tecla N: cambian la luz del ambiente.
- **21 farolas** junto a las intersecciones; su luz ilumina el suelo y los edificios cercanos y disminuye con la distancia.
- **Dos faros** luz concentrada que siguen al auto, con iluminación hacia adelante y alcance limitado, se encienden y apagan con F.
- **Luz difusa** calculada con las normales de cada cara, más reflejos y niebla. Las bombillas brillan con emisión propia, distinta de la luz que recibe el entorno.
- **Las luces de freno** se encienden al frenar.



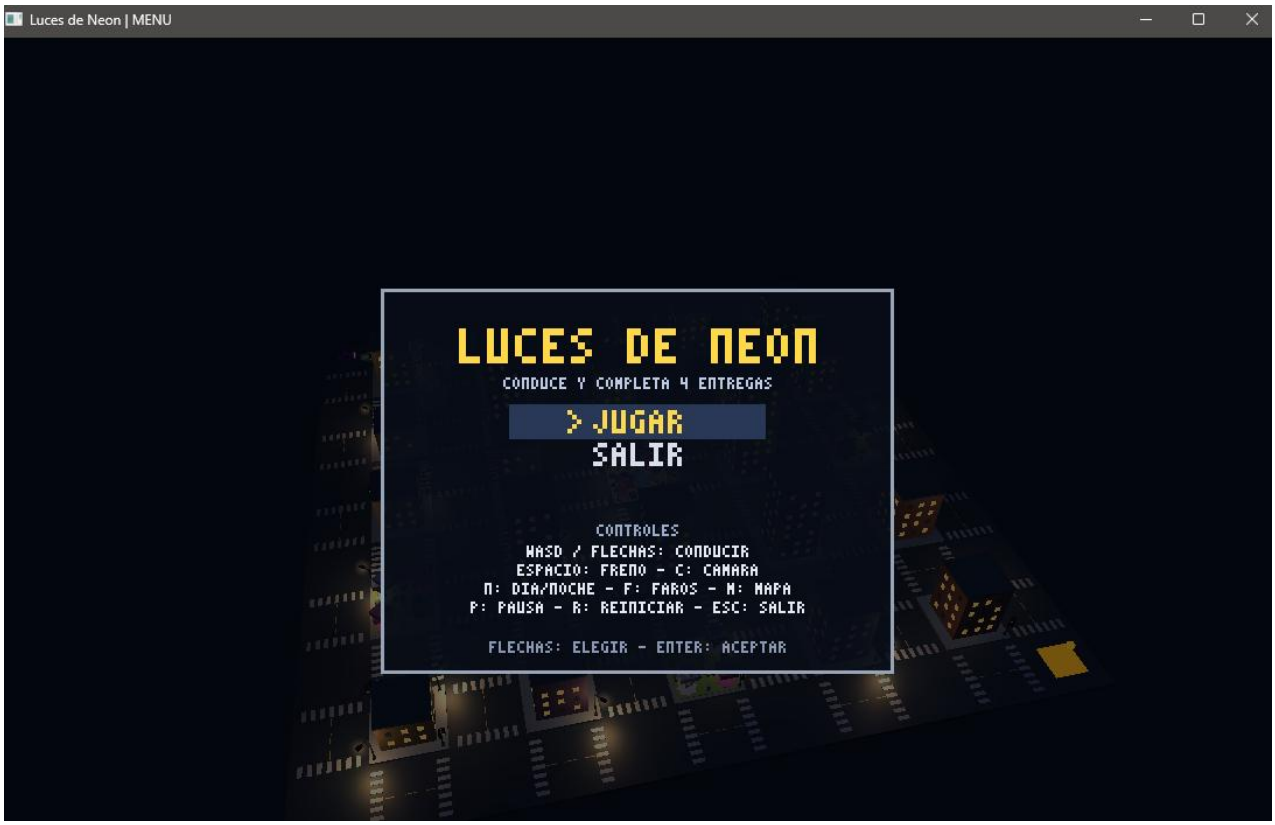
Elementos urbanos

- **Parques** con árboles (rosado, tajibo blanco y árbol verde) y un banco.
- **Plaza central** con piso de Piedra, Fuente de agua, 4 faroles que se encienden de noche, árboles y bancos.
- **Ventanas** en los edificios: oscuras de día y algunas habitaciones encendidas de noche.
- **Pasos peatonales** en los extremos de cada tramo de calle, junto a los cruces.
- **18 semáforos** con la secuencia rojo, verde, amarillo controlada por tiempo (ciclo de 12 segundos).
- **4 carteles de sectores** en las avenidas alrededor del Centro, que indican el sector hacia el que se va y se iluminan de noche.

Juego e interfaz

- **Minimapa** en una esquina: muestra toda la ciudad, la posición y orientación del auto, el destino activo y la N del norte. Mantiene su encuadre al redimensionar la ventana.
- **Tráfico autónomo:** vehículos celeste, amarillo y verde recorren rutas por el carril derecho, giran en las esquinas, respetan los cruces y se detienen si el jugador está delante.

- **Panel en pantalla** con sector actual, velocidad en km/h, barra de velocidad y estado de faros, día/noche, freno y minimapa.
- **Menú de inicio y pausa** con portada, opciones y ayuda de controles, P pausa todo el juego.





4. Organización del código

El proyecto se divide en cuatro clases que se heredan en cadena; cada una agrega una responsabilidad sin modificar las anteriores:

Clase	Responsabilidad
clase1	Ventana, ciclo del juego, shaders, mapa y edificios
clase2	Conducción: teclado, física del auto, colisiones y cámaras
clase3	Iluminación: día/noche, farolas, faros y halos
clase4	Funciones adicionales: entregas, decoración, semáforos, tráfico, minimapa, panel, menús y carteles

Todos los objetos se dibujan con cajas: cada caja recibe su posición, su tamaño y su color. Todo el movimiento usa deltaTime, por lo que el auto avanza lo mismo a 30 o a 144 cuadros por segundo. El proyecto incluye 29 pruebas automáticas (mvn test) que verifican el mapa, las colisiones, las luces, los semáforos, el tráfico y los menús.

5. Limitaciones

- Toda la geometría está hecha con cajas: no hay modelos 3D importados, texturas ni sonido.
- No hay sombras: la luz de una farola puede atravesar un edificio.
- Los semáforos son solo visuales: no detienen al jugador ni al tráfico.
- No hay peatones y las ruedas no giran.
- La fuente del panel y los menús es de píxeles y no tiene tildes ni eñes.
- Requiere una tarjeta gráfica compatible con OpenGL 3.3.

6. Cómo ejecutarlo

Requisitos: JDK 17 o superior y Maven. Abrir una terminal en la carpeta del proyecto (donde está pom.xml) y ejecutar:

```
mvn compile exec:exec
```

7. Recursos externos

- **Proyecto base:** AppCamara.java y AppLaberinto.java, proporcionados por el docente.
- **Java 17 y Maven:** lenguaje y compilación.
- **LWJGL 3.3.3:** GLFW (ventana y teclado) y OpenGL 3.3 Core (dibujo con shaders GLSL, VAO y VBO).
- **JUnit:** para las pruebas automáticas.